

Game Programming

Assignment 02



Group Members:

Saad Ali (55673)
Mahmood Khan (56067)
Muhammad Roshaan Idrees (56177)
Ahmed Tariq (56274)
Umer Farooq (56154)

Submitted to:

Mr. Abdul Mateen

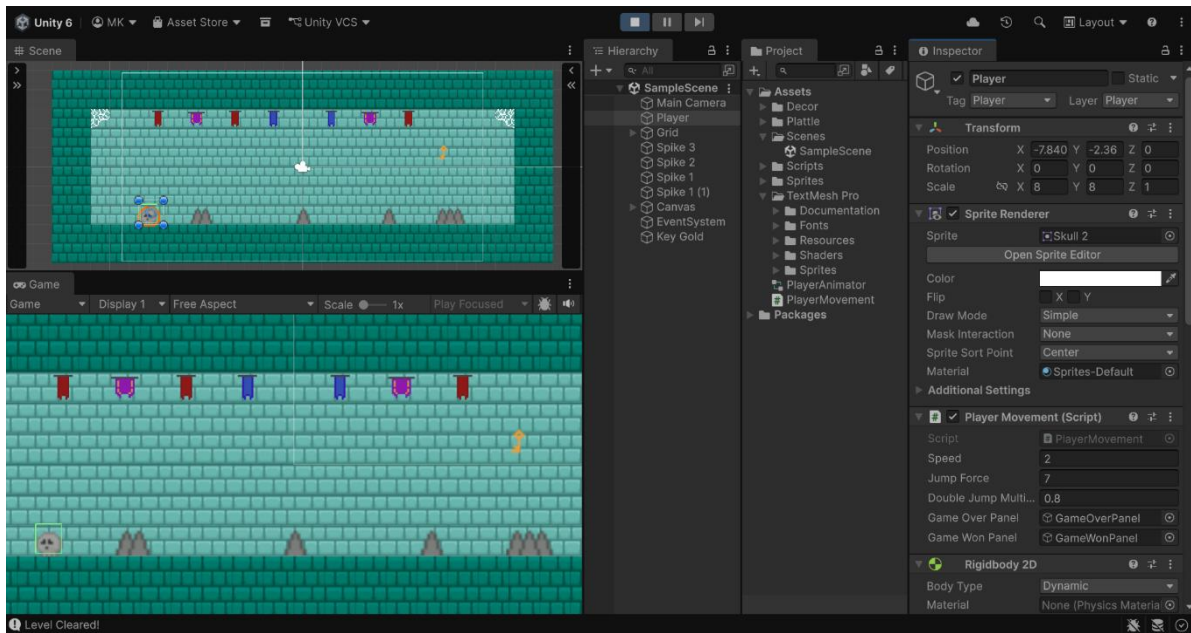
RIPHAH International University

Spring 2026

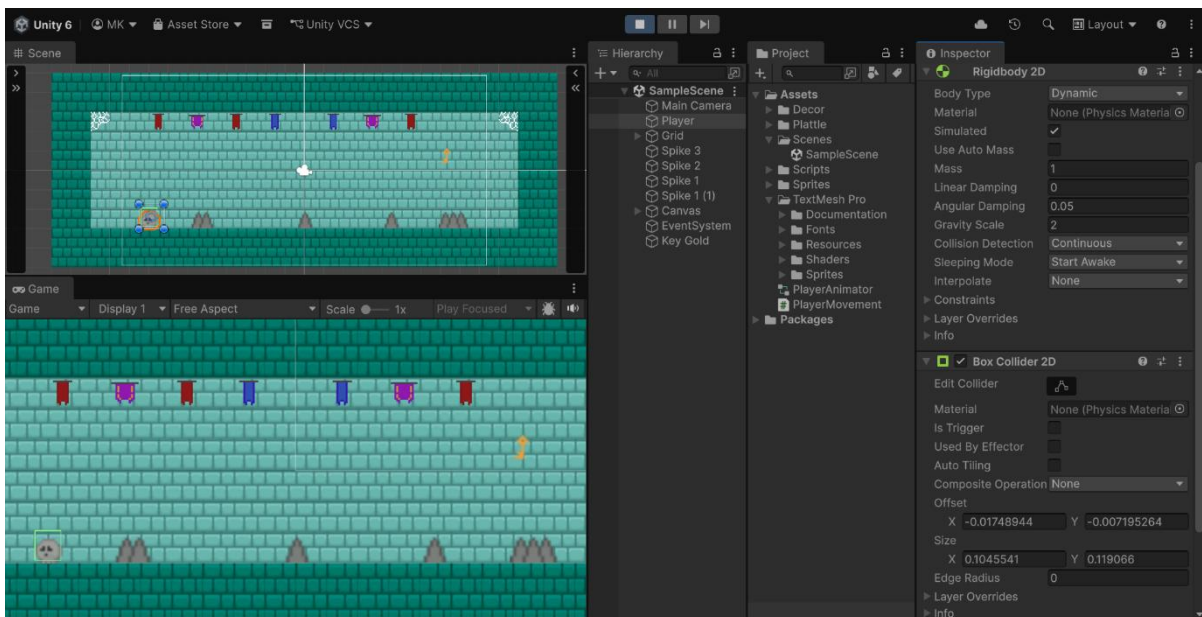
Faculty of Computing



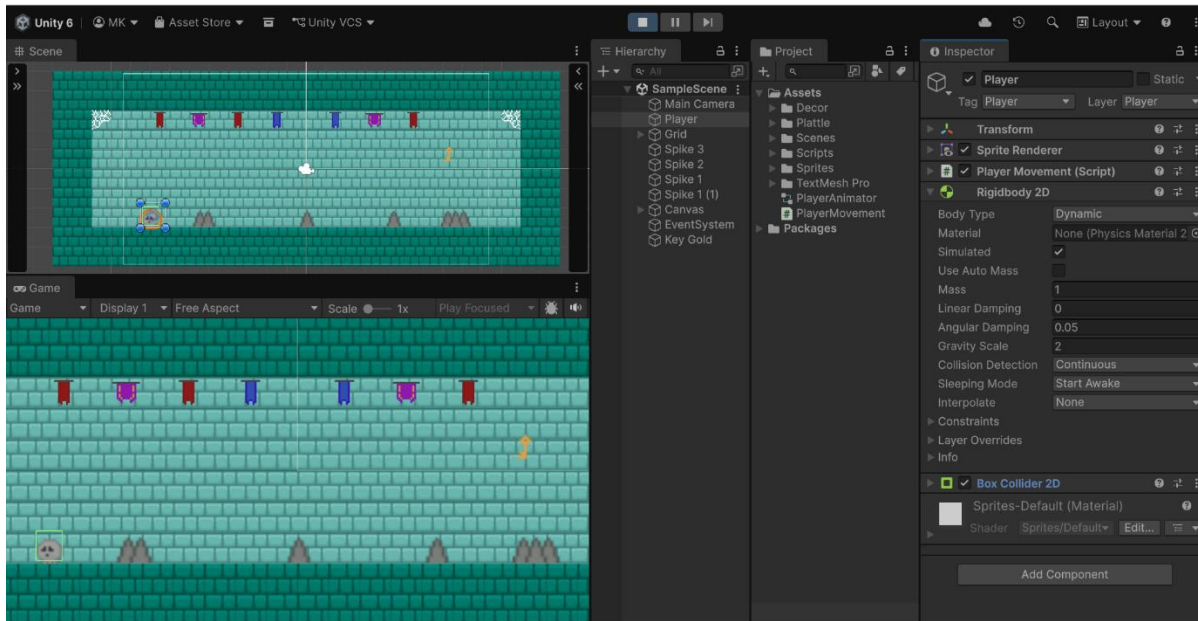
Unity Scene Overview – Player Character Setup



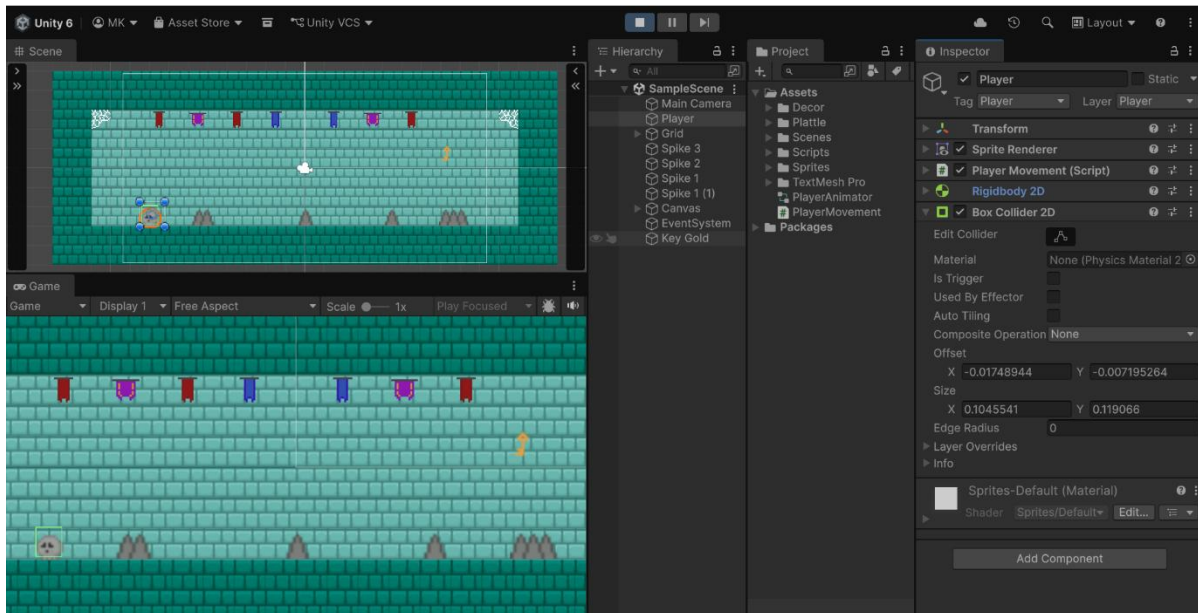
Player Character Inspector – All Components



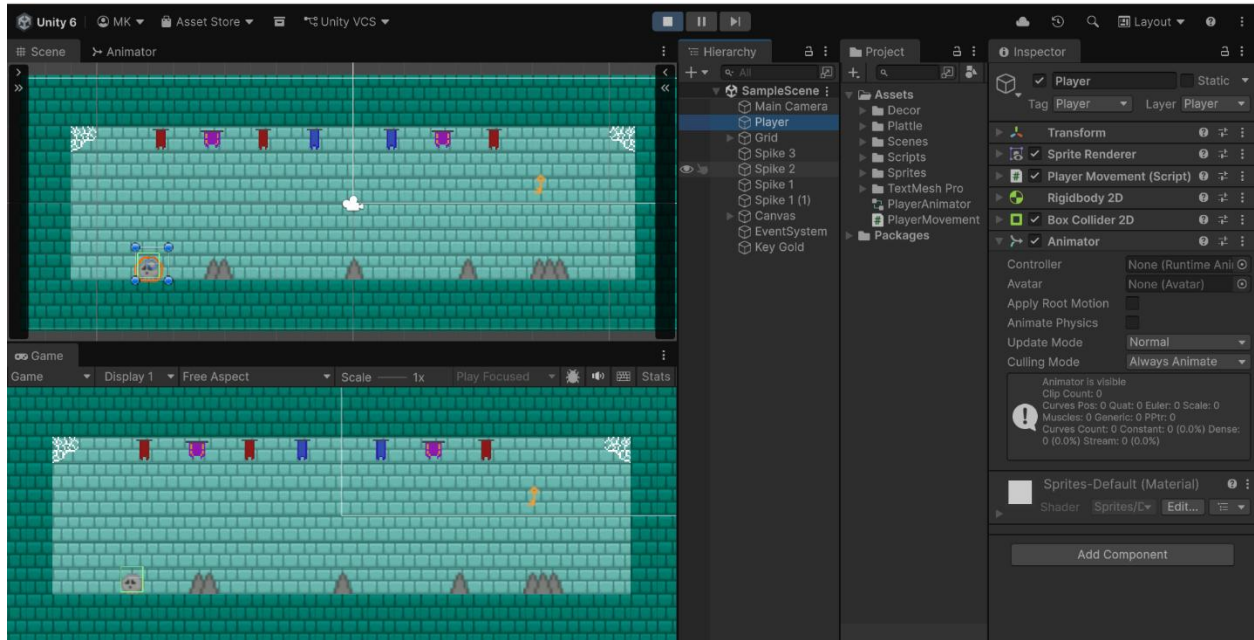
Rigidbody2D Component Configuration



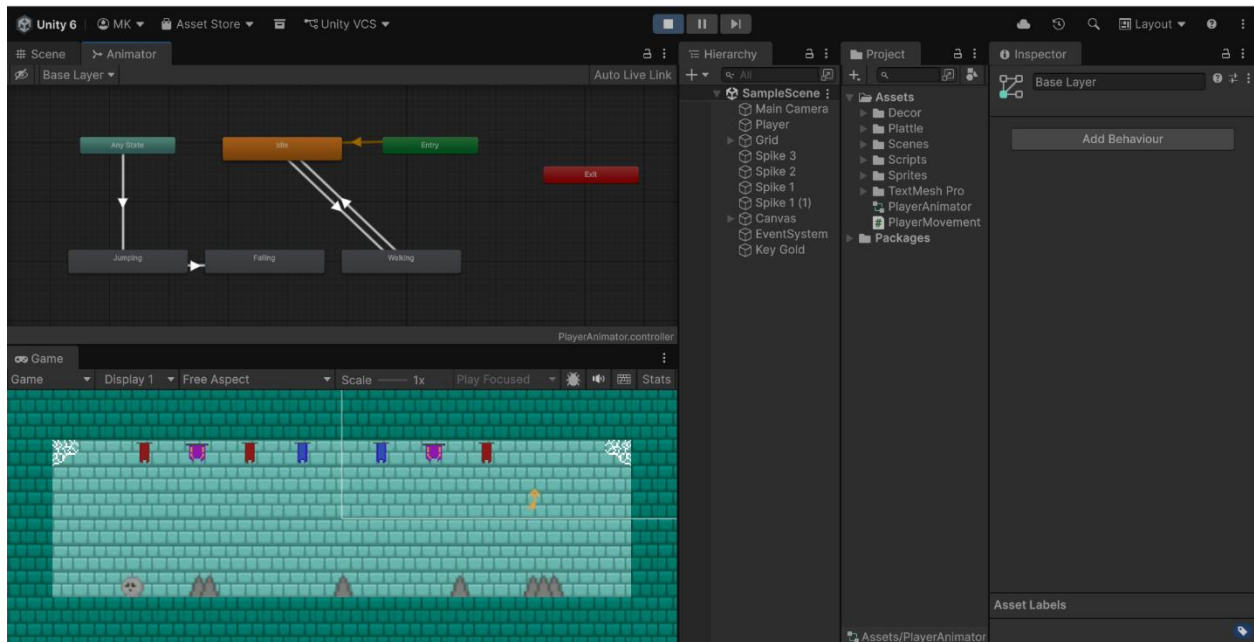
Box Collider 2D Component Configuration



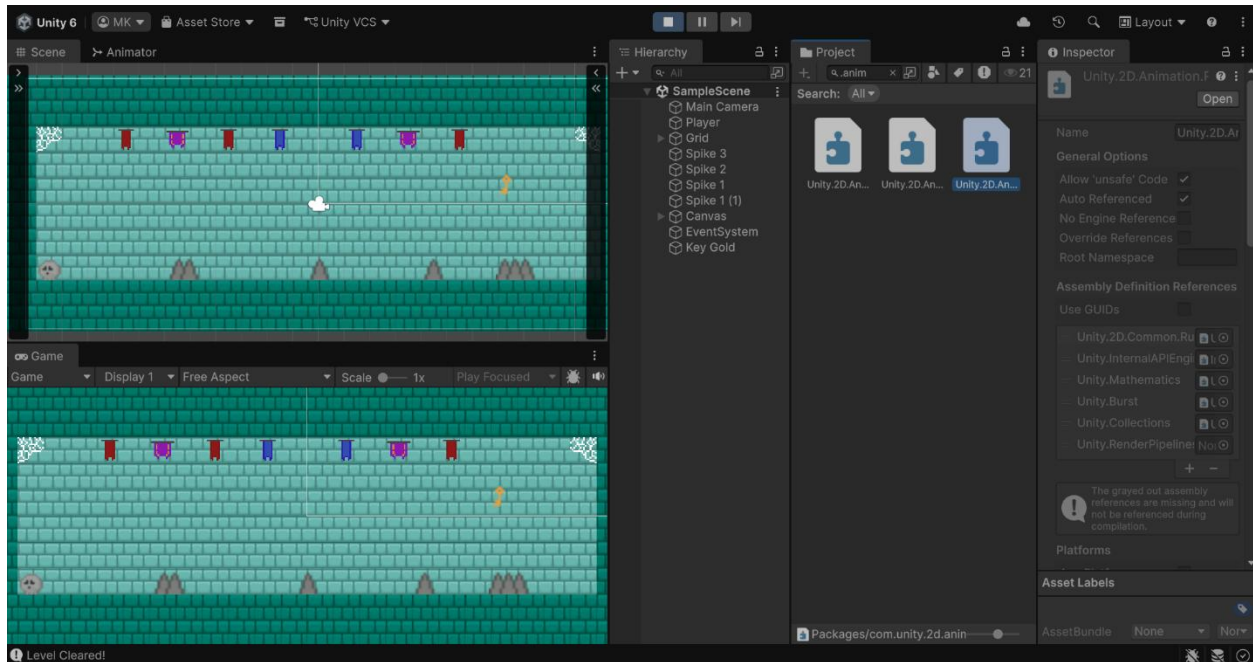
Animator Component Configuration



State Machine Diagram Component



Animation Files Configuration Page



Player State Machine Code

```
PlayerStateMachine.cs X
Assets > Scripts > PlayerStateMachine.cs > ...
1 //CODE FOR PLAYER STATE MACHINE IN UNITY
2 using System.Collections;
3 using System.Collections.Generic;
4 using UnityEngine;
5
6 9 references | @ Unity Script
7 public class PlayerStateMachine : MonoBehaviour
8 {
9     [Header("Movement Settings")]
10     4 references | @ Unity Serialized Field
11     public float speed = 5f;
12     2 references | @ Unity Serialized Field
13     public float jumpForce = 7f;
14     1 reference | @ Unity Serialized Field
15     public float doubleJumpMultiplier = 0.8f;
16
17     [Header("Camera Bounds")]
18     2 references | @ Unity Serialized Field
19     public float leftBound = -11f;
20     2 references | @ Unity Serialized Field
21     public float rightBound = 11.20f;
22     2 references | @ Unity Serialized Field
23     public float bottomBound = -3f;
24     2 references | @ Unity Serialized Field
25     public float topBound = 3.5f;
26
27     // Components
28     30 references | @ Unity Serialized Field
29     [HideInInspector] public Rigidbody2D rb;
30     29 references | @ Unity Serialized Field
31     [HideInInspector] public Animator animator;
32     1 reference | @ Unity Serialized Field
33     [HideInInspector] public SpriteRenderer spriteRenderer;
34 }
```



```
PlayerStateMachine.cs X
Assets > Scripts > PlayerStateMachine.cs > ...
6 public class PlayerStateMachine : MonoBehaviour
24 // State Machine
11 references
25 private PlayerBaseState currentState;
4 references
26 private PlayerStateFactory states;
27
28 // Public properties for states
5 references | @ Unity Serialized Field
29 [HideInInspector] public bool isGrounded;
5 references | @ Unity Serialized Field
30 [HideInInspector] public bool canDoubleJump;
11 references | @ Unity Serialized Field
31 [HideInInspector] public float moveInput;
5 references | @ Unity Serialized Field
32 [HideInInspector] public float playerWidth;
5 references | @ Unity Serialized Field
33 [HideInInspector] public float playerHeight;
34
0 references | @ Unity Message
35 void Start()
36 {
37 // Get components
38 rb = GetComponent<Rigidbody2D>();
39 animator = GetComponent<Animator>();
40 spriteRenderer = GetComponent<SpriteRenderer>();
41
42 // Calculate player bounds
43 CalculatePlayerBounds();
44
45 // Initialize states
46 states = new PlayerStateFactory(this);
```

```
PlayerStateMachine.cs X
Assets > Scripts > PlayerStateMachine.cs > ...
6 public class PlayerStateMachine : MonoBehaviour
35 void Start()
48 // Start with idle state
49 currentState = states.Idle();
50 currentState.Enter();
51
52 if (rb == null)
53 Debug.LogError("No Rigidbody2D on Player!");
54
55
1 reference
56 void CalculatePlayerBounds()
57 {
58 SpriteRenderer spriteRenderer = GetComponent<SpriteRenderer>();
59 if (spriteRenderer != null)
60 {
61 playerWidth = spriteRenderer.bounds.extents.x;
62 playerHeight = spriteRenderer.bounds.extents.y;
63 }
64 else
65 {
66 Collider2D collider = GetComponent<Collider2D>();
67 if (collider != null)
68 {
69 playerWidth = collider.bounds.extents.x;
70 playerHeight = collider.bounds.extents.y;
71 }
72 else
73 {
74 playerWidth = 0.5f;
75 playerHeight = 0.5f;
76 }
```



```
PlayerStateMachine.cs X
Assets > Scripts > PlayerStateMachine.cs > ...
6 public class PlayerStateMachine : MonoBehaviour
  0 references | 0 Unity Message
  void Update()
  {
    // Get input
    moveInput = Input.GetAxisRaw("Horizontal");

    // Update current state
    currentState.Update();

    // Keep player in bounds
    KeepInCameraBounds();
  }

  10 references
  public void ChangeState(PlayerBaseState newState)
  {
    if (currentState != null)
      currentState.Exit();

    currentState = newState;
    currentState.Enter();
  }

  1 reference
  void KeepInCameraBounds()
  {
    Vector3 playerPos = transform.position;
    Vector3 newPos = playerPos;
    bool clamped = false;

    // X bounds
    float minX = leftBound + playerWidth;
```

```
PlayerStateMachine.cs X
Assets > Scripts > PlayerStateMachine.cs > ...
6 public class PlayerStateMachine : MonoBehaviour
  101 void KeepInCameraBounds()
  107 // X bounds
  108 float minX = leftBound + playerWidth;
  109 float maxX = rightBound - playerWidth;
  111 if (playerPos.x < minX)
  112 {
  113     newPos.x = minX;
  114     if (rb.linearVelocity.x < 0) rb.linearVelocity = new Vector2(0, rb.linearVelocity.y);
  115     clamped = true;
  116 }
  117 else if (playerPos.x > maxX)
  118 {
  119     newPos.x = maxX;
  120     if (rb.linearVelocity.x > 0) rb.linearVelocity = new Vector2(0, rb.linearVelocity.y);
  121     clamped = true;
  122 }
  124 // Y bounds - MODIFIED: Don't force grounded at top
  125 float minY = bottomBound + playerHeight;
  126 float maxY = topBound - playerHeight;
  128 if (playerPos.y < minY)
  129 {
  130     newPos.y = minY;
  131     if (rb.linearVelocity.y < 0)
  132     {
  133         rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0);
  134     }
  135     clamped = true;
  136 }
```

```
PlayerStateMachine.cs X
Assets > Scripts > PlayerStateMachine.cs > ...
6 public class PlayerStateMachine : MonoBehaviour
  101 void KeepInCameraBounds()
  137 else if (playerPos.y > maxY)
  138 {
  139     newPos.y = maxY;
  140     if (rb.linearVelocity.y > 0)
  141     {
  142         rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0);
  143     }
  144     clamped = true;
  145 }
  147 if (clamped)
  148     transform.position = newPos;
  150 }

  0 references | 0 Unity Message
  void OnCollisionEnter2D(Collision2D collision)
  {
    if (collision.gameObject.CompareTag("Ground"))
    {
      isGrounded = true;
      canDoubleJump = false;

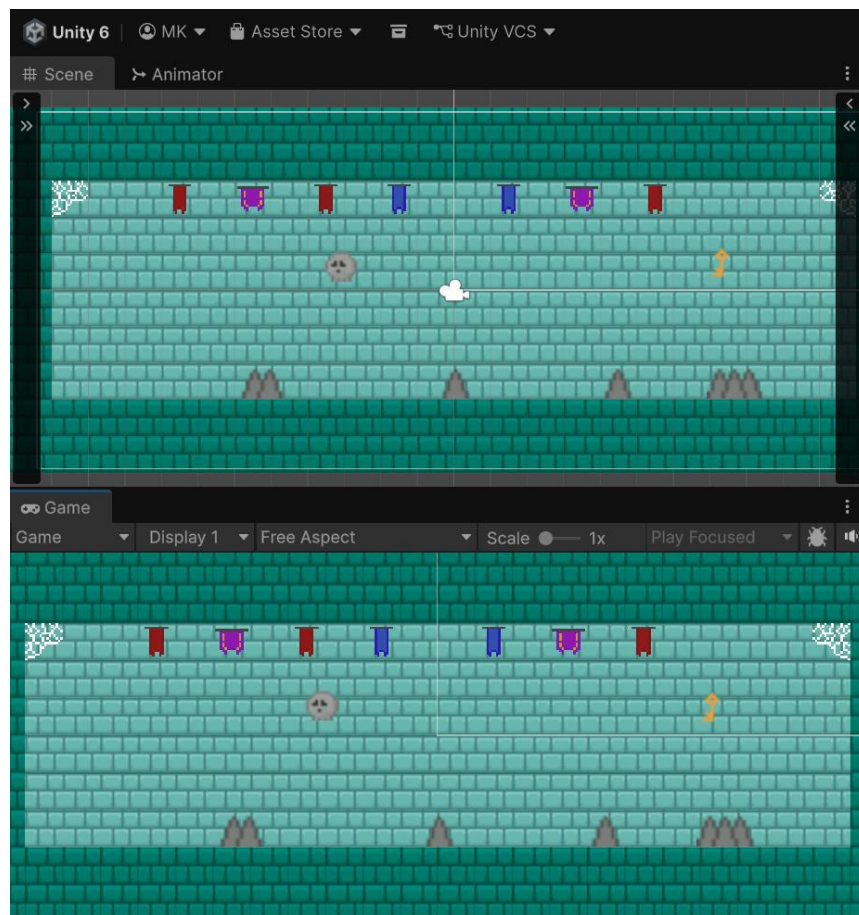
      Debug.Log("Touched ground! Current state: " + currentState.GetType().Name);

      // Only change state if we're in an air state
      if (currentState is JumpingState || currentState is FallingState || currentState is DoubleJumpState)
      {
        if (Mathf.Abs(moveInput) > 0)
        {
          Debug.Log("Switching to Walking after landing");
        }
      }
    }
  }
```



```
PlayerStateMachine.cs
Assets > Scripts > PlayerStateMachine.cs > ...
6 public class PlayerStateMachine : MonoBehaviour
151 void OnCollisionEnter2D(Collision2D collision)
165 {
166     Debug.Log("Switching to Walking after landing");
167     ChangeState(states.Walking());
168 }
169 else
170 {
171     Debug.Log("Switching to Idle after landing");
172     ChangeState(states.Idle());
173 }
174 }
175 }
176
0 references [Unity Message]
177 void OnCollisionExit2D(Collision2D collision)
178 {
179     if (collision.gameObject.CompareTag("Ground"))
180     {
181         isGrounded = false;
182     }
183 }
184
0 references [Unity Message]
185 void OnDrawGizmos()
186 {
187     Gizmos.color = Color.green;
188     Vector3 bottomLeft = new Vector3(leftBound, bottomBound, 0);
189     Vector3 topRight = new Vector3(rightBound, topBound, 0);
190     Vector3 size = topRight - bottomLeft;
191     Gizmos.DrawWireCube(bottomLeft + size / 2, size);
192 }
```

Gameplay Demo – Character Movement in Action



Collision Detection – Player Blocked in Collider

